

UNIVERSIDAD PRIVADA DEL VALLE

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO DE SISTEMAS I

APLICACIÓN DE ESTÁNDARES DE DESARROLLO BAJO METODOLOGÍA XP

Paralelo: “B”

Estudiantes: Jose Alejandro Salazar Quiroga

Melani Quintela Aguilar

Julio Mateo Bustillos Serrano

Jose Martin Leaño Mercado

Docente: Ing.

La Paz 24 de septiembre de 2026

Gestión II – 2026

INDICE

1. DIAGNÓSTICO DEL PROYECTO 1

1.1. Lenguajes de Programación Utilizados 1

1.2. Frameworks y Tecnologías Empleadas 1

1.2.1. Servicios y Lógica del Sistema 1

1.2.2. Base de Datos 2

1.3. Herramientas de Gestión Seleccionadas 2

1.4. Problemas potenciales derivados de la ausencia de estándares 2

1.4.1. Diferencias al escribir el código del sistema 2

1.4.2. Falta de uniformidad en el diseño de la base de datos 3

1.4.3. Conflictos y pérdida de archivos al usar Git 3

1.4.4. Mensajes poco claros al registrar avances 3

1.4.5. Falta de sincronización entre el diseño visual y la lógica del sistema 3

1.5. Riesgos para la calidad del software. 4

1.5.1. Riesgo de información rota o desordenada en MySQL 4

1.5.2. Riesgo de fallas en la detección de phishing 4

1.5.3. Riesgo de lentitud al probar y corregir errores 4

1.5.4. Riesgo de retrasar las entregas en cada iteración 4

2. Manual de Estándares 5

2.1. Estándares de Codificación 5
2.1.1. Nombres en PHP para la lógica del servidor: 5
2.1.2. Nombres en JavaScript para las pantallas web 5
2.1.3. Nombres en las hojas de estilo CSS 5
2.1.4. Reglas generales de orden y limpieza 6
2.2. Estándares de Carpetas 6
2.3. Estándares de Base de Datos 9
2.3.1. Nombres descriptivos en tablas y columnas 9
2.3.2. Claves primarias normalizadas 9
2.3.3. Claves foráneas y relaciones 9
2.3.4. Restricciones de integridad 10
2.4. Estándares de Git 10
2.4.1. Convenciones de commits 10
2.4.2. Estrategia de ramas 11
2.4.3. Frecuencia de integración 11
2.5. Estándares de Documentación 12
2.5.1. Estructura de Historias de Usuario 12
2.6. Casos de Uso 12
2.6.1. Estándares para Diagramas 13

2.6.2. Manuales Técnicos y Archivo README 13

3. INTEGRACIÓN DE ESTÁNDARES EN XP 13

3.1. Matriz de Relación entre Estándares y Prácticas XP 13

4. APLICACIÓN PRÁCTICA 13

4.1. Historia de Usuario Actualizada y Sprint Backlog en Jira 14

4.2. Estándares de Base de Datos Aplicados (MySQL) 14

4.3. Estándares de Programación Aplicados 15

4.4. Estándares de Git Aplicados 16

1. DIAGNÓSTICO DEL PROYECTO

1.1. LENGUAJES DE PROGRAMACIÓN UTILIZADOS

- **PHP:** Es el lenguaje utilizado en el servidor para el portal web. Se encarga de procesar las solicitudes de inicio de sesión, verificar credenciales, gestionar el acceso a las cuentas y consultar la información de análisis e historiales para mostrarlos en el navegador.
- **JavaScript:** Se utiliza en la página web para brindar interactividad visual en la pantalla. Permite realizar validaciones inmediatas en los formularios de acceso antes de enviarlos, y controlar animaciones visuales en los paneles de estadísticas e historial.
- **SQL:** Es el lenguaje de base de datos utilizado para estructurar las tablas, registrar usuarios, guardar los enlaces analizados y consultar los reportes dentro del sistema de almacenamiento relacional

1.2. FRAMEWORKS Y TECNOLOGÍAS EMPLEADAS

- **HTML5:** Permite estructurar las vistas del portal web mediante bloques claros y ordenados, como por ejemplo la cabecera, el menú lateral y la zona principal de contenido.
- **CSS3:** Se encarga del diseño visual y la apariencia del portal web. Utiliza reglas modernas de cuadrícula y cajas flexibles para que las pantallas se adapten tanto a computadoras de escritorio como a teléfonos móviles, manteniendo una identidad visual oscura con verde neón orientada a la ciberseguridad.

1.2.1. Servicios y Lógica del Sistema

- **Detección y Expansión de Enlaces Acortados:** Es la lógica encargada de recibir enlaces de servicios que acortan direcciones web para rastrear su destino real antes de abrirlos, evitando que el usuario sea redirigido a sitios dañinos sin darse cuenta.

- **Integración con Servicios y APIs de Verificación:** Permite consultar servicios externos especializados en reputación web para comprobar si una dirección URL o un dominio ya fue reportado como peligroso en otras partes del mundo.

1.2.2. Base de Datos

- **MySQL:** Es el motor de base de datos relacional seleccionado para almacenar y ordenar toda la información del proyecto. Permite vincular a los usuarios con sus análisis realizados, sus reportes y los niveles de riesgo mediante tablas relacionadas entre sí con claves numéricas únicas.

1.3. HERRAMIENTAS DE GESTIÓN SELECCIONADAS

- **Tablero de Gestión de Tareas en Jira**: Se utiliza Jira como herramienta de tablero visual para organizar el conjunto de historias de usuario y dividirlas según las cinco iteraciones de trabajo planificadas. Esto permite que el equipo sepa qué tareas están pendientes, cuáles están en desarrollo y cuáles ya fueron probadas.
- **Git y GitHub:** Git es el sistema utilizado para registrar cada cambio realizado en los archivos del proyecto, mientras que GitHub actúa como el repositorio central en la nube donde el equipo sube y sincroniza el código del portal web de forma compartida.
- **Herramientas de Diseño y Modelado:** Se emplean herramientas de diseño para elaborar los bocetos de la página web antes de programarlas, así como programas de diagramado como ERD plus para dibujar el modelo de tablas y relaciones de la base de datos MySQL.

1.4. PROBLEMAS POTENCIALES DERIVADOS DE LA AUSENCIA DE ESTÁNDARES

1.4.1. Diferencias al escribir el código del sistema

Al repartirse las tareas de programación entre la parte visual, la lógica del sistema y el acceso a los datos, la falta de reglas provoca que se mezclen estilos muy distintos. Por

ejemplo, combinar palabras en inglés con español, usar abreviaciones confusas o estructurar las funciones de forma desordenada. Esto dificulta que cualquier integrante pueda leer, revisar o corregir el trabajo de otro compañero con rapidez.

1.4.2. Falta de uniformidad en el diseño de la base de datos

Si no se define una norma obligatoria para nombrar tablas, columnas y relaciones en MySQL, se cometen errores graves. Por ejemplo, llamar a la identificación del usuario como id_usuario en una tabla y en otra llamarlo simplemente codigo_usuario o usuario_id. Esta falta de orden provoca que las consultas fallen al momento de guardar análisis de URLs o reportes de enlaces sospechosos.

1.4.3. Conflictos y pérdida de archivos al usar Git

En el repositorio del proyecto, la ausencia de una estrategia de ramas definida provocaría que los integrantes suban cambios directamente sobre la rama principal. Esto genera el riesgo de sobrescribir archivos trabajados por otro compañero, por ejemplo, al actualizar los formularios de inicio de sesión o las pantallas de estadísticas, causando pérdida de tiempo y retrasos en el calendario de trabajo.

1.4.4. Mensajes poco claros al registrar avances

Guardar cambios en el repositorio con mensajes vagos, por ejemplo, escribiendo solo actualización, cambios o fix, no permite saber qué tarea específica se completó ni a qué historia de usuario pertenece. Esto impide rastrear con precisión el origen de un error si el sistema comienza a fallar después de una integración.

1.4.5. Falta de sincronización entre el diseño visual y la lógica del sistema

Si el diseño de las pantallas no sigue una estructura fija de estilos y clases, al momento de programar la lógica los componentes visuales pierden consistencia. Esto provoca que los botones, las alertas de seguridad y las tarjetas de resultados se vean desalineados o no respondan de la misma manera en la página web.

1.5. RIESGOS PARA LA CALIDAD DEL SOFTWARE.

1.5.1. Riesgo de información rota o desordenada en MySQL

Si no se definen reglas estrictas de integridad en las tablas de la base de datos, un usuario podría eliminarse dejando sus análisis o reportes sueltos y sin dueño, lo que provocaría errores en el cálculo de las estadísticas personales y en el historial.

1.5.2. Riesgo de fallas en la detección de phishing

Click Seguro tiene como objetivo principal evitar el robo de credenciales mediante el análisis de enlaces. Si las funciones encargadas de procesar las URLs ingresadas por texto no están estandarizadas, un enlace peligroso podría catalogarse por equivocación como seguro, poniendo en peligro la información del usuario final.

1.5.3. Riesgo de lentitud al probar y corregir errores

Durante las etapas de prueba, el equipo debe verificar que cada función cumpla con lo prometido en las historias de usuario. Si el código no está ordenado bajo una misma estructura, encontrar y resolver el origen de una falla toma el doble de tiempo, retrasando las correcciones necesarias.

1.5.4. Riesgo de retrasar las entregas en cada iteración

La metodología de trabajo se basa en entregar avances funcionales de forma continua y en periodos cortos. Si el equipo pasa días resolviendo choques de archivos en Git o corrigiendo desajustes entre la base de datos y la interfaz, no se podrá cumplir a tiempo con las entregas planificadas en el proyecto.

2. MANUAL DE ESTÁNDARES

2.1. ESTÁNDARES DE CODIFICACIÓN

2.1.1. Nombres en PHP para la lógica del servidor:

- Los nombres de los archivos y las clases principales se escriben con la primera letra de cada palabra en mayúscula, por ejemplo UsuarioController, AuthService o ConexionBD.
- Los nombres de las funciones se escriben comenzando con minúscula y usando mayúscula en las siguientes palabras, expresando siempre una acción clara, por ejemplo validarCredenciales, registrarUsuario o consultarEstadisticas.
- Las variables deben describir con claridad lo que guardan, por ejemplo correoUsuario, passwordPlano o estadoCuenta. Se prohíbe el uso de variables con nombres de una sola letra, como por ejemplo x o p.
- Los valores constantes que nunca cambian se escriben completamente en mayúsculas separadas por guion bajo, por ejemplo DB_HOST o MAX_INTENTOS_LOGIN.

2.1.2. Nombres en JavaScript para las pantallas web

- Las variables y constantes se definen usando let o const, quedando prohibido el uso de var.
- Las funciones que responden a acciones del usuario comienzan indicando el evento correspondiente, por ejemplo al presionar un botón se usa handleSubmitLogin o al escribir en un campo se usa onPasswordInput.

2.1.3. Nombres en las hojas de estilo CSS

- Se aplica una convención de nombres basada en bloques visuales para no mezclar estilos entre pantallas distintas.
- Se usa el nombre del contenedor principal seguido de dos guiones bajos para sus partes internas, por ejemplo login-box__title o form-group__input.

- Para cambios de estado o color se usan dos guiones medios, por ejemplo alert--error para mensajes en rojo o alert--success para avisos en verde.

2.1.4. Reglas generales de orden y limpieza

- Si una función hace varias tareas a la vez, debe dividirse en funciones más pequeñas y simples.
- Se permite un máximo de dos niveles de condiciones anidadas para evitar código enredado y difícil de leer.
- La separación del código debe mantener cuatro espacios de sangría en PHP y dos o cuatro espacios uniformes en la parte web. No se permite mezclar tabulaciones con espacios.
- Los comentarios en el código solo se utilizan para explicar por qué se tomó una decisión de seguridad o un cálculo especial, evitando comentarios obvios que solo repitan lo que el código ya dice.

2.2. ESTÁNDARES DE CARPETAS

Para ordenar los archivos del repositorio oficial en GitHub y evitar que todos los documentos queden sueltos en una sola carpeta, se establece la siguiente estructura por capas de trabajo:

```
ClickSeguroEstadisticas/

├── config/      # Conexión a la base de datos y variables globales

│   └── database.php            # Configuración de conexión PDO a MySQL

├── controllers/  # Controladores que procesan las peticiones del portal web

│   ├── AuthController.php # Control de inicio de sesión y registro de usuario

│   ├── AnalysisController.php   # Control de ingreso manual, análisis y reanálisis de URLs

│   ├── StatsController.php  # Control de datos del panel de estadísticas personales
```

```
|   ├── HistoryController.php    # Control de consulta y eliminación del historial
|   └── ReportController.php     # Control del registro de URLs sospechosas reportadas
├── models/        # Modelos que consultan directamente las tablas de MySQL
|   ├── Usuario.php             # Consultas a la tabla usuario
|   ├── Analisis.php# Consultas a las tablas analisis, detalle_analisis y riesgo
|   ├── Url.php                 # Consultas a las tablas url y dominio
|   └── Reporte.php             # Consultas a la tabla reporte
├── public/        # Recursos accesibles directamente desde el navegador web
|   ├── css/                    # Hojas de estilo del sistema web
|   |   ├── global.css# Estilos generales, fondo de rejilla y línea de escáner
|   |   ├── auth.css            # Estilos para inicio de sesión y registro
|   |   ├── analysis.css # Estilos para las pantallas de análisis y resultado
|   |   ├── dashboard.css # Estilos para el panel de estadísticas personales
|   |   └── historial.css    # Estilos para las tablas de historial y detalle
|   ├── js/                     # Scripts de soporte visual para la interfaz
|   |   ├── auth.js # Validaciones visuales en formularios de acceso
|   |   ├── analysis.js# Pegado rápido de URLs y validación del campo de texto
|   |   ├── dashboard.js# Efectos visuales en tarjetas y contadores de números
|   |   └── historial.js  # Confirmaciones visuales antes de eliminar registros
|   └── img/                    # Logotipos, escudos e iconos de la interfaz
```

```
├── views/          # Pantallas del portal web según los mockups del proyecto
│   ├── auth/                   # Pantallas de autenticación y acceso
│   │   ├── login.php           # Pantalla 2: Inicio de sesión web
│   │   └── register.php        # Pantalla 1: Registro de nueva cuenta
│   ├── analysis/               # Pantallas del flujo de análisis de enlaces
│   │   ├── analizar.php        # Pantalla 3: Ingreso y pegado manual de URL
│   │   └── resultado.php  # Pantalla 4: Resultado del análisis y clasificación
│   ├── dashboard/              # Pantallas de consulta y gestión del usuario
│   │   ├── estadisticas.php    # Pantalla 5: Panel de estadísticas personales
│   │   ├── historial.php       # Pantalla 6: Historial de análisis realizados
│   │   └── detalle_url.php     # Pantalla 7: Detalle y reanálisis de una URL
│   └── layouts/                # Componentes comunes de la interfaz
│       ├── header.php          # Barra superior con logo y datos de la cuenta
│       └── sidebar.php         # Menú lateral con opciones de navegación
├── database/     # Scripts de creación y respaldo de la base de datos
│   ├── ddl_schema.sql # Creación oficial de las tablas y sus relaciones en
MySQL
│   └── seeds.sql    # Catálogos de tipos de riesgo, tipos de origen y amenazas
├── docs/         # Documentación del proyecto, backlog de Jira e historias
└── README.md         # Instrucciones técnicas de instalación y despliegue
```

2.3. ESTÁNDARES DE BASE DE DATOS

2.3.1. Nombres descriptivos en tablas y columnas

- Todas las tablas deben nombrarse en minúsculas, en singular y usando palabras completas en español. Por ejemplo, se deben usar nombres como usuario, url, dominio, analisis, reporte, tipo_riesgo, tipo_origen, tipo_amenaza, analisis_amenaza, detalle_analisis y fuente_verificacion. Se prohíbe el uso de plurales como usuarios o reportes.
- Los nombres de las columnas se escriben en minúsculas separando las palabras con guion bajo. Por ejemplo, fecha_registro, resultado_general, direccion_url o nombre_dominio. Quedan prohibidas las abreviaturas confusas como por ejemplo fec_reg o dir_u.

2.3.2. Claves primarias normalizadas

- Cada tabla debe contar con una clave primaria numérica única que identifique cada registro de forma irrepetible.
- La clave primaria debe nombrarse obligatoriamente con el prefijo id_ seguido del nombre de la tabla en singular. Por ejemplo, id_usuario, id_url, id_dominio, id_analisis, id_reporte, id_tipo_riesgo, id_tipo_origen, id_tipo_amenaza, id_detalle y id_fuente.
- Todas las claves primarias numéricas se configuran como enteros autoincrementales para que el motor MySQL asigne el número correlativo de manera automática.

2.3.3. Claves foráneas y relaciones

Las columnas que conectan una tabla con otra deben llamarse exactamente igual que la clave primaria a la que hacen referencia. Por ejemplo, en la tabla analisis, la columna que apunta a la tabla usuario debe llamarse id_usuario, la que apunta a la tabla url se llama id_url, y las que apuntan a los catálogos se llaman id_tipo_riesgo y id_tipo_origen.

2.3.4. Restricciones de integridad

- **No permitir campos vacíos**: Todos los datos indispensables para el sistema deben definirse como no nulos
- **Datos únicos:** Los campos que no deben repetirse en el sistema deben configurarse con la regla de unicidad
- **Protección contra borrado accidental:** Para las tablas de categorías fijas, como por ejemplo tipo_riesgo o tipo_origen, se configura una regla de restricción de borrado para que el sistema impida eliminar una categoría si existen análisis que dependen de ella.
- **Eliminación controlada de datos personales:** Para cumplir con la historia de usuario sobre eliminación del historial a petición del usuario, cuando una cuenta es dada de baja voluntariamente por el titular con su debida confirmación, sus registros asociados en analisis y reporte se depuran de forma automática y en cascada para no dejar datos huérfanos en el sistema.

2.4. ESTÁNDARES DE GIT

2.4.1. Convenciones de commits

Los mensajes al guardar cambios en el repositorio deben seguir una estructura estandarizada que explique con claridad qué se modificó y en qué parte del proyecto. Cada mensaje se compone de un tipo en minúsculas, el área afectada y una descripción breve redactada en tiempo presente:

- **feat:** Se utiliza cuando se agrega una funcionalidad nueva al sistema, como feat: agregar formulario de registro en la vista web.
- **fix:** Se emplea para corregir un fallo o error en el código, como fix: corregir validacion de contraseñas coincidentes en inicio de sesion.
- **style:** Se usa para cambios visuales, hojas de estilo o diseño que no alteran la lógica del sistema, como style: aplicar tema oscuro con acentos neon al panel de estadisticas.

- **refactor:** Se aplica cuando se reordena o limpia el código existente sin agregar funciones nuevas ni corregir errores.
- **docs:** Se destina a modificaciones en la documentación del proyecto o en el archivo README.
- **database:** Se utiliza cuando se agregan o actualizan scripts de tablas y datos en MySQL.

2.4.2. Estrategia de ramas

Queda prohibido trabajar o subir cambios directos sobre la rama principal main. La organización del código se divide en tres niveles de ramas:

Rama main: Contiene únicamente código estable, probado y listo para su presentación o entrega final.

Rama develop: Es la rama común de integración continua donde se reúnen los avances de las distintas tareas antes de pasar a la versión final.

Ramas por historia de usuario: Cada nueva tarea o pantalla se desarrolla en una rama temporal independiente que lleva el identificador asignado en Jira y una descripción corta

- feature/HU-13-inicio-sesion-web
- feature/HU-13-registro-usuario
- feature/HU-15-panel-estadisticas
- feature/HU-04-historial-analisis

2.4.3. Frecuencia de integración

- Cada integrante debe registrar sus avances en commits pequeños y bien definidos, evitando acumular demasiado trabajo sin guardar.
- Las ramas de tareas específicas no deben permanecer abiertas por más de dos días sin integrarse a la rama develop.

- Para unir una rama de tarea con la rama develop, es obligatorio abrir una solicitud de revisión en GitHub. Otro integrante del equipo debe revisar el código y verificar que no genere errores antes de aprobar la integración definitiva.

2.5. ESTÁNDARES DE DOCUMENTACIÓN

2.5.1. Estructura de Historias de Usuario

Todas las historias de usuario registradas en Jira y en la documentación oficial deben mantener el formato estándar de la metodología XP:

- **Identificador único:** Código numérico de seguimiento, como HU-01, HU-06 o HU-13. Nombre descriptivo: Título corto que sintetice la tarea asignada.
- **Prioridad y estimación**: Clasificación de importancia (Alta o Media) y esfuerzo estimado medido en puntos de historia usando la escala 1, 2, 3, 5 y 8.
- **Enunciado de valor:** Redacción obligatoria bajo la estructura:
  Como: tipo de usuario.
  Quiero: funcionalidad solicitada.
  Para: objetivo o beneficio obtenido.
- **Criterios de aceptación:** Condiciones comprobables que determinan cuándo la historia está terminada y lista para ser validada.
- **Tareas técnicas**: Lista de actividades concretas de diseño, programación, base de datos y pruebas necesarias para completarla.

2.6. CASOS DE USO

En las funciones principales donde interactúan la página web o la aplicación móvil con los servicios del sistema, se redacta una descripción que detalle:

- Actor que inicia la acción, como el usuario o el administrador.
- Precondiciones necesarias, como tener una cuenta registrada o haber iniciado sesión.

- Flujo principal de pasos desde que se ingresa el dato hasta que se muestra el resultado en pantalla.
- Flujos alternativos en caso de error, como ingresar una URL con formato inválido o credenciales incorrectas.

2.6.1. Estándares para Diagramas

- **Diagrama Entidad-Relación:** Se elabora bajo la notación estándar de pata de gallo, indicando con claridad las claves primarias con el prefijo id_, las claves foráneas, los tipos de datos y la cardinalidad de cada enlace entre tablas.
- **Diagramas de Arquitectura:** Deben representar la división formal en tres niveles: la capa de presentación (web en PHP, HTML y CSS), la capa de lógica del sistema (servicios de análisis y APIs de verificación) y la capa de persistencia (MySQL).

2.6.2. Manuales Técnicos y Archivo README

El repositorio en GitHub debe incluir un archivo README.md que funcione como guía técnica para cualquier persona que requiera instalar y probar el proyecto. Este documento debe contener:

- Descripción general del sistema y problema que resuelve.
- Requisitos previos de software, indicando las versiones compatibles de PHP, MySQL y herramientas de desarrollo.
- Pasos ordenados para clonar el repositorio, configurar el archivo de base de datos y ejecutar el proyecto de forma local.
- Cuentas de usuario de prueba para verificar el acceso inmediato al sistema web.

3. INTEGRACIÓN DE ESTÁNDARES EN XP

3.1. MATRIZ DE RELACIÓN ENTRE ESTÁNDARES Y PRÁCTICAS XP

4. APLICACIÓN PRÁCTICA

4.1. HISTORIA DE USUARIO ACTUALIZADA Y SPRINT BACKLOG EN JIRA

- **Identificador de Historia**: HU-13
- **Título:** Inicio de sesión en el sistema web
- **Prioridad en el Negocio:** Alta
- **Puntos de Historia Estimados**: 5
- **Iteración Asignada en Jira:** Primera iteración
- **Enunciado de Valor:**
  Como: Usuario del portal web
  Quiero*:* Ingresar mis credenciales de acceso
  Para: Acceder a mi panel de estadísticas personales y consultar mi historial de análisis
- **Criterios de Aceptación:**

El formulario debe exigir obligatoriamente el ingreso de correo electrónico y contraseña.

Si las credenciales no existen en la base de datos o son erróneas, el sistema debe desplegar un mensaje visual de error sin limpiar el campo de correo.

Si las credenciales son correctas, el sistema debe inicializar la sesión en el servidor y redirigir de forma automática al panel de estadísticas.

Sprint Backlog Asociado:

- **Tarea 1:** Diseñar la interfaz web con temática de ciberseguridad en HTML5 y CSS3.
- **Tarea 2:** Implementar la tabla usuario con restricciones de unicidad en MySQL.
- **Tarea 3:** Construir el modelo Usuario.php con consultas preparadas contra inyección SQL.
- **Tarea 4:** Programar el controlador AuthController.php con validaciones de contraseña cifrada.
- **Tarea 5:** Ejecutar pruebas de acceso con credenciales válidas e inválidas.

4.2. ESTÁNDARES DE BASE DE DATOS APLICADOS (MYSQL)

Se aplican nombres en minúsculas, claves primarias normalizadas con el prefijo id_ y restricciones estrictas de integridad sobre la entidad usuario:

```sql
-- Creación estandarizada de la tabla usuario
CREATE TABLE usuario (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(150) NOT NULL UNIQUE,
    contrasena VARCHAR(255) NOT NULL,
    fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) DEFAULT 'ACTIVO'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Consulta estandarizada para autenticación segura en el login
SELECT id_usuario, nombre, correo, contrasena, estado
FROM usuario
WHERE correo = ? AND estado = 'ACTIVO'
LIMIT 1;
```

4.3. ESTÁNDARES DE PROGRAMACIÓN APLICADOS

El fragmento implementado en controllers/AuthController.php respeta la regla de nombres descriptivos en camelCase, límite de anidamiento y uso de sentencias preparadas para seguridad informática:

```php
<?php
// Archivo: controllers/AuthController.php

class AuthController {
    private $conexionBD;

    public function __construct($db) {
        $this->conexionBD = $db;
    }

    public function iniciarSesion($correo, $contrasenaPlana) {
        if (empty($correo) || empty($contrasenaPlana)) {
            return "Todos los campos son obligatorios.";
        }

        $sentencia = $this->conexionBD->prepare(
            "SELECT id_usuario, nombre, contrasena FROM usuario WHERE
correo = ? LIMIT 1"
        );
        $sentencia->bind_param("s", $correo);
        $sentencia->execute();
        $resultado = $sentencia->get_result();

        if ($resultado->num_rows === 0) {
            return "Credenciales de acceso inválidas.";
        }

        $usuario = $resultado->fetch_assoc();
```

```
        if (!password_verify($contrasenaPlana, $usuario['contrasena'])) {
            return "Credenciales de acceso inválidas.";
        }

        $_SESSION['id_usuario'] = $usuario['id_usuario'];
        $_SESSION['nombre_usuario'] = $usuario['nombre'];

        return "EXITO";
    }
}
```

4.4. ESTÁNDARES DE GIT APLICADOS

Registro ordenado del flujo de trabajo en la terminal para el desarrollo de esta funcionalidad en el repositorio oficial de GitHub:

```
# 1. Creación y cambio a la rama asignada según la historia de usuario
git checkout -b feature/HU-13-inicio-sesion-web

# 2. Confirmación de la estructura visual del formulario (metodología BEM)
git add views/auth/login.php public/css/auth.css
git commit -m "style(auth): maquetar formulario de inicio de sesion con
estetica terminal"

# 3. Confirmación de la lógica del controlador de autenticación
git add controllers/AuthController.php
git commit -m "feat(auth): implementar logica de inicio de sesion y
verificacion de hash"

# 4. Envío de la rama para revisión de código antes de integrar a develop
git push origin feature/HU-13-inicio-sesion-web
```